

Modelo de Atores Fora de Sistemas Distribuídos

por Jorge Harrisonn Mantovanelli Thomes Vieira

Modelo de Atores

Teoria matemática originalmente concebida por Carl Hewitt em 1973 para sistemas distribuídos e massivamente concorrentes.

O Modelo de Atores enxerga sistemas computacionais em termos de unidades autônomas chamadas de **atores** que se comunicam entre si exclusivamente através de **mensagens** assíncronas.

A comunicação, feita via passagem de mensagens, elimina a necessidade de compartilhamento de memória e *locks*. Este padrão serviu de base para linguagens de programação como Erlang e Elixir, tornando-se o paradigma central dessas linguagens.

Objetivo

Este trabalho realiza uma experimentação aplicando o Modelo de Atores em um contexto distante do seu domínio natural: **aplicações centralizadas em Rust**. Rust não possui o Modelo de Atores como paradigma central, diferentemente de linguagens como Elixir.

O objetivo é entender o esforço necessário para aplicar um padrão arquitetural fora do seu contexto natural e quais consequências isso traz para o projeto. Busca-se produzir um arcabouço para permitir a aplicação do padrão fora do ambiente original.

Estudo de Caso: Patch-Hub

Patch-Hub é uma ferramenta de terminal desenvolvida como parte do ecossistema KWorkflow para interagir com a API do Lore Kernel Archive. A ferramenta permite aos desenvolvedores navegar por listas de patches do kernel Linux, aplicar filtros de busca, visualizar conteúdo de patches e gerenciar seu fluxo de trabalho de revisão.

Para validação, foi realizada uma reescrita do projeto aplicando o Modelo de Atores com as adaptações propostas.

Arquitetura

Testabilidade

A abordagem facilita imensamente a criação de testes unitários. Como cada ator possui responsabilidades bem definidas, é fácil criar mocks para suas dependências.

A cobertura de testes aumentou cerca de 3x, permitindo testes paralelos sem condições de corrida.

Extensibilidade

Adicionar novos atores e mensagens é simples seguindo padrões bem definidos. Novos atores seguem uma estrutura consistente, e novas mensagens são facilmente integradas ao esquema sem grandes modificações estruturais.

Complexidade

A base de código aproximadamente **dobrou** de tamanho comparada à implementação original. Isso é um indicativo de que a abordagem tende a produzir código mais verboso, exigindo cuidados especiais para manter qualidade e legibilidade a longo prazo.

Conclusão

O principal resultado demonstra que a adequação de padrões arquiteturais pode ser **surpreendentemente flexível**. Mesmo utilizando um padrão aparentemente inapropriado para o contexto, foi possível extrair valor significativo da abordagem ao reimaginar tanto o problema quanto o padrão, permitindo que ambos se encaixassem de forma interessante.

Conclui-se que, embora os padrões arquiteturais possuam domínios naturais de aplicação, eles **não estão rigidamente limitados** a esses contextos. A exploração criativa de padrões em domínios distantes de sua origem pode resultar em soluções bem-sucedidas, desde que haja disposição para adaptar e reimaginar as abordagens estabelecidas. Naturalmente, esse processo de adaptação demanda planejamento cuidadoso e múltiplas iterações.